



Ch03-Activity I-Hacking Password

1.

- **Objective:** Understand how attackers use pre-built word lists (dictionaries) to crack hashes of common passwords.
- **Scenario:** You have discovered a SHA-1 hash in a compromised system:
d54cc1fe76f5186380a0939d2fc1723c44e8a5f7.

You suspect the password is a simple, common word, possibly with some character substitutions.

- **Task:** Write a Python program that reads a list of words, applies common substitutions, hashes the result, and checks if it matches the target hash. Note that you might want to include substitution in your code (lowercase, uppercase, number for letter ['o' ⇒ 0, 'l' ⇒ 1, 'i' ⇒ 1]).
- **Answer:**

```
Password is : ThaiLanD
```

◦ Code

```
import hashlib
import bcrypt

with open("10k-most-common.txt", "r") as f:
    words = f.read().splitlines()

Answer = "d54cc1fe76f5186380a0939d2fc1723c44e8a5f7"

def Recursive(word, index):
```

```

    if index == len(word):
        SHA1 = hashlib.sha1(bytes(word, "utf-8")).hexdigest()

        # print("word is: ", word)
        if SHA1 == Answer:
            print("Password is : ", word)
            return

    # Upper case
    wordUpper = word[:index] + word[index:index+1].upper()
    Recursive(wordUpper, index + 1)

    # Lower case
    wordLower = word[:index] + word[index:index+1].lower()
    Recursive(wordLower, index + 1)

    # Special case
    if word[index].lower() == 'o':
        wordSpecial = word[:index] + '0' + word[index+
1:]
        Recursive(wordSpecial, index+1)
    if word[index].lower() == 'i' or word[index].lower()
== 'l':
        wordSpecial = word[:index] + '1' + word[index+
1:]
        Recursive(wordSpecial, index+1)

for word in words:
    Recursive(word, 0)

```

2.

- **Objective:** To understand *why* modern password hashing algorithms like bcrypt are more secure than older ones like MD5 and SHA-1.
- **Task:** Design and run an experiment to measure how many hashes each algorithm can compute in a fixed amount of time. The code must test

atleast **MD5** , **SHA-1** , and **bcrypt** .(You may also try additional algorithms like SHA256, SHA512, scrypt, Argon2.)

- Hint: Use time function in python.
- Answer:
 - 100 first Password of `10k-most-common.txt` for run test.

```
MD5: 0.000123s (816327 hashes/second)
SHA1: 0.000095s (1054852 hashes/second)
SHA256: 0.000093s (1069519 hashes/second)
SHA512: 0.000113s (886525 hashes/second)
BCRYPT: 19.066341s (5 hashes/second)
SCRYPT: 6.170428s (16 hashes/second)
ARGON2: 4.620328s (22 hashes/second)
```

- Code

```
import hashlib
import bcrypt
import time
from argon2 import PasswordHasher

ph = PasswordHasher()

with open("10k-most-common.txt", "r") as f:
    words = f.read().splitlines()

max_round = 100

def Hash(word, hash_type):
    start_time = time.perf_counter()
    for index, word in enumerate(words):
        if index == max_round:
            break

        if hash_type == "SHA1":
            hashlib.sha1(bytes(word, "utf-8")).hexdigest
        ()

        elif hash_type == "MD5":
```

```

        hashlib.md5(bytes(word, "utf-8")).hexdigest
    ()
    elif hash_type == "BCRYPT":
        bcrypt.hashpw(bytes(word, "utf-8"), bcrypt.g
    ensalt())
    elif hash_type == "SHA256":
        hashlib.sha256(bytes(word, "utf-8")).hexdigest()
    elif hash_type == "SHA512":
        hashlib.sha512(bytes(word, "utf-8")).hexdigest()
    elif hash_type == "SCRYPT":
        hashlib.scrypt(bytes(word, "utf-8"), salt=
    b"salt", n=2**14, r=8, p=1)
    elif hash_type == "ARGON2":
        ph.hash(word)
    elapsed_time = time.perf_counter() - start_time
    return elapsed_time

algorithms = ["MD5", "SHA1", "SHA256", "SHA512", "BCRYPT", "SCRYPT", "ARGON2"]

for algorithm in algorithms:
    elapsed_time = Hash(words, algorithm)
    print(f"{algorithm}: {elapsed_time:.6f}s ({max_round/elapsed_time:.0f} hashes/second)")

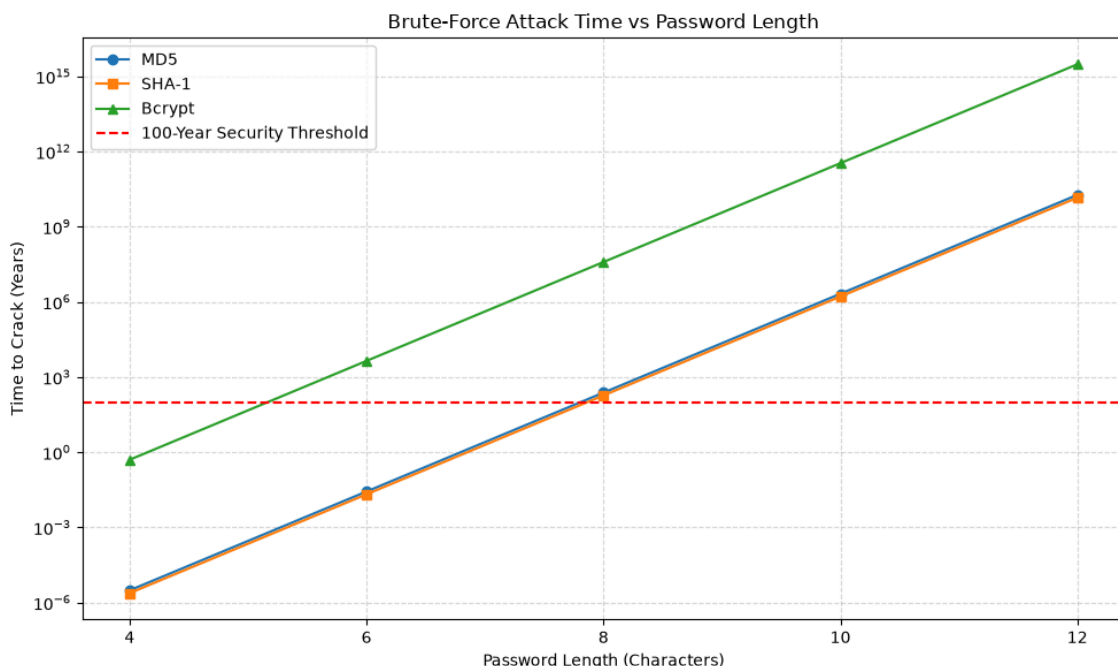
```

3.

- **Objective:** To apply the performance measurements to understand the importance of password length and algorithm choice.
- **Task:** Based on the measurements from Exercise 2, estimate how long it would take an attacker to brute-force a password of a given length. You may assume that the password contains only upper-case, lower-case, numbers and symbols. What does it suggest about the length of a proper password. (ie. Use more than a year to brute force.)

- **Answer:**

- กำหนดสมการ $T = \frac{k^n}{R}$
 - T : เวลาที่ต้องใช้
 - R : อัตราความเร็วในการคำนวณได้กี่ password ต่อ 1 second (hash/second)
 - n : ความยาวของรหัสผ่าน
 - k : จำนวน Character ที่เป็นไปได้
 - $k = 26(\text{upper-case letters}) + 26(\text{lower-case letters}) + 10(\text{numbers}) + 32(\text{symbols / special characters}) = 94$
 - k^n : ให้เป็น Worst case โดย Hacker password ยาว n ได้ในรูปแบบสุดท้าย (hash)
 - เพิ่มเติม: R ที่ได้จากทดลอง Password ที่ขนาดไม่คงที่ สมมุติให้พอเทียบเท่ากับการทำ hash function ของ password ยาว n เนื่องจาก hash function ทำงานเป็น block ทำให้ทำงานแทบจะเท่ากันเมื่อขนาดต่างกัน
- Case 100 Year — จะเห็นได้ว่า MD5, SHA-1 จะต้องใช้อย่างน้อย 8 Character ส่วน Bcrypt ใช้เพียง 6 Character ก็ใช้เวลามากกว่า 100 ปีแล้ว



4.

- **Question:** If a given hash value is from a bcrypt algorithm, is it practical to do a brute-force attack?
- **Answer:** Not practical เพราะ ใช้เวลานานมากในการ Brute Force จากข้อ 3. จะเห็นได้ว่าเพียง 4 Character ก็ใช้เวลาถึง 1 ปี และโดยทั่วไป password ขึ้นต่ำก็ 8 Character แล้วแม้มี Computeสูงก็ตามที่

5.

- **Question:** If a given hash value is from a bcrypt algorithm, is it practical to perform a rainbow table attack?
- **Answer:** Not practical เพราะ แม้ rainbow table อาจลด k^n ไปเยอะมาก แต่ยังติดตรง salt ที่ Randomอีกชั้นที่ต้องใช้เวลาในการประมาณมหาศาลอยู่ดี

6.

- **Question:** You have to store a password in a database. Please explain your design/strategy for securely storing it.(Hints: Proper hash function, Salting, Cost factor, Database Security)
- **Answer:**
 - **Proper Hash Function:** ไม่เก็บ passwordเป็น plaintext และ ควรใช้Algorithmสมัยใหม่ (ไม่ใช่รุ่นเก่า MD5 หรือ SHA-1) ex bcrypt, scrypt, Argon2
 - **Salting:** ใช้ salt แบบสุ่มสำหรับแต่ละ user ก่อน hash
 - **Cost Factor:** Cost Factor สูงๆแต่ยัง Server response user ได้เร็วอยู่
 - **Database Security:** มี Authorization access database, มี rate limit, มี multi-factor authentication